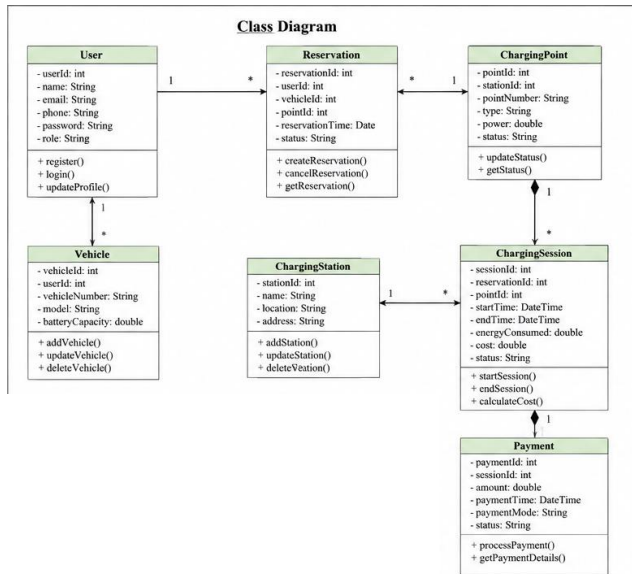


ASSIGNMENT

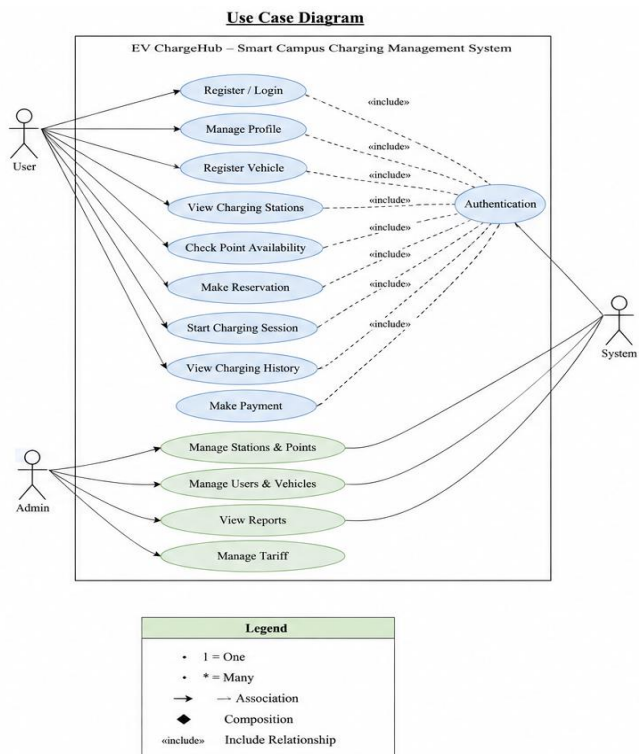
Problem Statement :

Design, implement, and evaluate a GUI-based Java application using AWT/Swing and JDBC to manage an E-Vehicle Charging Facility for a large University campus, where multiple charging points are distributed across campus locations and manual coordination can lead to charging-point conflicts, waiting time, inefficient utilization, and difficulty in maintaining accurate charging records. The prototype shall automate the charging workflow — registering electric vehicles and users, maintaining charging stations and charging points, checking point availability, reserving charging slots, recording vehicle check-in and check-out, creating and updating charging sessions, recording charging duration and energy consumed, computing charging cost using configured tariff rates, maintaining charging and payment records, and generating station-utilization, energy-consumption, and usage reports. The core entities (users, vehicles, charging stations, charging points, reservations, charging sessions, energy usage, and payments) shall be maintained through appropriate relational database tables with suitable primary and foreign key relationships to ensure data integrity and consistency. The graphical user interface shall use appropriate AWT/Swing controls, layout managers, menus, tables, and event-handling mechanisms to support data entry, navigation, reservation, charging operations, searching, filtering, validation, and report generation. Charging-point status such as available, occupied, reserved, and maintenance shall be displayed appropriately through the GUI. JDBC shall be used to establish connectivity with a MySQL database and perform SQL operations such as insert, select, update, and delete using suitable statement types including Statement, PreparedStatement, and CallableStatement. Robustness shall be ensured through input validation and exception handling for invalid data, duplicate records, unavailable charging points, conflicting reservations, database connectivity failures, and SQL exceptions. The completed prototype shall demonstrate the integration of GUI design, event-driven programming, layout management, AWT/Swing controls, JDBC connectivity, relational database design, SQL operations, validation, exception handling, charging-cost computation, and management reporting to address an industry-specific smart-campus electric-vehicle charging requirement.

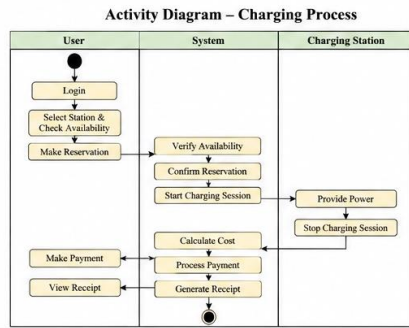
CLASS DIAGRAM :



CASE DIAGRAM :



ACTIVITY DIAGRAM :



PSEUDO CODE :

Application Startup

BEGIN

Load / initialize application data

Create the Swing user interface

Configure window size, theme, sidebar and main body

Display Dashboard

END

Dashboard Generation

BEGIN

totalPoints ← 0

available ← 0

occupied ← 0

FOR each station

FOR each charging point in station

totalPoints ← totalPoints + 1

IF point.status = "Available"

available ← available + 1

END IF

IF point.status = "Occupied"

occupied ← occupied + 1

END IF

END FOR

END FOR

energy ← SUM(session.kWh for all sessions)

Display totalPoints

- Display available
- Display occupied
- Display energy
- Display live point-status cards
- Display quick actions

END

Create Reservation

BEGIN

IF no vehicles exist

- Show "Register a vehicle first"

- STOP

END IF

Find points whose status is Available or Reserved

IF no usable point exists

- Show "No usable charging points available"

- STOP

END IF

Select vehicle

Select charging point

Enter slot

IF user confirms

- Create Reservation record

- Add reservation to reservation list

- Change selected point status to Reserved

- Save state

- Refresh Reservations page

END IF

END

Record Charging Session

BEGIN

IF no vehicle exists

- Show "Register a vehicle first"

- STOP

END IF

Build list of usable points
Select vehicle and charging point
Enter energyConsumed and duration

IF values are invalid
 Show validation warning
 STOP
END IF

selectedPoint.status ← Available
cost ← energyConsumed × 8.50

Create Session(vehicle, point, energyConsumed, duration, Completed)
Add Session to session list

Create Payment(session, cost, Paid)
Add Payment to payment list

Save state
Refresh Charging Sessions page
END

Analytics Calculation

BEGIN
 sessions ← number of recorded sessions
 energy ← SUM(session.kWh)
 revenue ← SUM(session.cost)

 Count points by status:
 Available
 Occupied
 Reserved
 Maintenance

 Calculate utilization from the operational point counts
 Display KPI cards
 Draw bar chart using the four status counts
 Display management insights
END

CODE :

Domain Classes

```
}
```

```
static class Vehicle {  
    int id;  
    int userId;  
    String reg;  
    String model;  
  
    Vehicle(int id, int userId, String reg, String model) {  
        this.id = id;  
        this.userId = userId;  
        this.reg = reg;  
        this.model = model;  
    }  
}
```

```
static class Point {  
    int id;  
    String status;  
    String type;  
  
    Point(int id, String status, String type) {  
        this.id = id;  
        this.status = status;  
        this.type = type;  
    }  
}
```

```
static class Station {  
    int id;  
    String name;  
    String location;  
    List<Point> points = new ArrayList<>();  
  
    Station(int id, String name, String location) {  
        this.id = id;  
        this.name = name;
```

```
        this.location = location;
    }
}
```

```
static class Reservation {
```

```
    int id;
    int vehicleId;
    int pointId;
    String slot;
    String status;
```

```
    Reservation(int id, int vehicleId, int pointId, String slot, String status) {
```

```
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.slot = slot;
        this.status = status;
```

```
    }
}
```

```
static class Session {
```

```
    int id;
    int vehicleId;
    int pointId;
    double kwh;
    double hours;
    double cost;
    String status;
```

```
    Session(int id, int vehicleId, int pointId, double kwh, double hours, String status) {
```

```
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.kwh = kwh;
        this.hours = hours;
        this.cost = kwh * TARIFF;
        this.status = status;
```

```
    }
}
```

```

static class Payment {
    int id;
    int sessionId;
    double amount;
    String status;

    Payment(int id, int sessionId, double amount, String status) {
        this.id = id;
        this.sessionId = sessionId;
        this.amount = amount;
        this.status = status;
    }
}

public static void main(String[] args) {
    load();
    SwingUtilities.invokeLater(Main::build);
}

static void build() {

    try {
        UIManager.setLookAndFeel(
            UIManager.getCrossPlatformLookAndFeelClassName()

```

Application Initialization

```

} catch (Exception ignored) {
}

frame = new JFrame(
    "EV ChargeHub | Smart Campus Charging"
);

frame.setDefaultCloseOperation(
    JFrame.EXIT_ON_CLOSE
);

frame.setSize(1250, 790);
frame.setMinimumSize(
    new Dimension(1050, 680)

```



```
);

frame.setLocationRelativeTo(null);

JPanel root = new JPanel(
    new BorderLayout()
);

root.setBackground(BG);

root.add(
    sidebar(),
    BorderLayout.WEST
);

JPanel main = new JPanel(
    new BorderLayout()
);

main.setBackground(BG);

main.add(
    topbar(),
    BorderLayout.NORTH
);

body = new JPanel(
    new BorderLayout()
);

body.setBackground(BG);

body.setBorder(
    new EmptyBorder(
        20,
        26,
        26,
        26
    )
);
```

```

);

main.add(
    body,
    BorderLayout.CENTER
);

root.add(
    main,
    BorderLayout.CENTER
);

frame.setContentPane(root);
frame.setVisible(true);

show("Dashboard");
}

static JPanel sidebar() {

    JPanel p = new JPanel(
        new BorderLayout()
    );

    p.setPreferredSize(
        new Dimension(225, 790)

```

Dashboard KPI Computation

```

"Across all stations",
    BLUE
    )
);

cards.add(
    card(
        "Available",
        String.valueOf(available),
        available + " ready to charge",
        GREEN
    )

```

```
);
```

```
cards.add(  
    card(  
        "Active / Occupied",  
        String.valueOf(occupied),  
        "Currently in use",  
        ORANGE  
    )  
);
```

```
cards.add(  
    card(  
        "Energy Delivered",  
        String.format(  
            "%.1f kWh",  
            energy  
        ),  
        "Recorded sessions",  
        PURPLE  
    )  
);
```

```
root.add(  
    cards,  
    BorderLayout.NORTH  
);
```

```
JPanel middle = new JPanel(  
    new GridLayout(  
        1,  
        2,  
        16,  
        0  
    )  
);
```

```
middle.setOpaque(false);
```

```
middle.add(  

```

```

        pointPanel()
    );

    middle.add(
        quickPanel()
    );

    root.add(
        middle,
        BorderLayout.CENTER
    );

    body.add(root);
}

```

```

static JPanel pointPanel() {

```

```

    JPanel p = panel();

```

Station Status Interaction

```

}

```

```

JTable t = table(m);

```

```

t.getColumnModel()
    .getColumn(5)
    .setCellRenderer(
        new ButtonRenderer()
    );

```

```

t.getColumnModel()
    .getColumn(5)
    .setCellEditor(
        new ButtonEditor(
            new JCheckBox(),
            row -> {

                int point =
                    Integer.parseInt(

```

```

        t.getValueAt(
            row,
            2
        )
        .toString()
        .replace(
            "P-",
            ""
        )
    );

    changeStatus(point);

    show("Stations");
}

)

);

root.add(
    new JScrollPane(t),
    BorderLayout.CENTER
);

body.add(root);
}

static void addStation() {

    JTextField name =
        new JTextField();

    JTextField location =
        new JTextField();

    JComboBox<String> type =
        new JComboBox<>(
            new String[]{
                "AC Type-2",
                "DC Fast",
                "CCS2"
            }
        );

```

```

        }
    );

    JPanel f = form();

    f.add(
        new JLabel(
            "Station Name"
        )
    );

    f.add(name);

    f.add(
        new JLabel(
            "Campus Location"
        )
    );

    f.add(location);

```

Cost Calculation

```

Session(int id, int vehicleId, int pointId, double kwh, double hours, String status) {
    this.id = id;
    this.vehicleId = vehicleId;
    this.pointId = pointId;
    this.kwh = kwh;
    this.hours = hours;
    this.cost = kwh * TARIFF;
    this.status = status;
}

```

Analytics Chart

```

root.add(
    middle,
    BorderLayout.CENTER
);

body.add(root);
}

```

```

static JPanel reportChart(
    int available,
    int occupied,
    int reserved,
    int maintenance) {

    JPanel p = panel();

    p.setLayout(
        new BorderLayout()
    );

    p.add(
        heading(
            "Point Distribution",
            "Current status of every charging point"
        ),
        BorderLayout.NORTH
    );

    JPanel chart = new JPanel() {

        protected void paintComponent(
            Graphics g) {

            super.paintComponent(g);

            int[] values = {
                available,
                occupied,
                reserved,
                maintenance
            };

            String[] names = {
                "Available",
                "Occupied",
                "Reserved",

```

```

        "Maintenance"
    };

    Color[] colors = {
        GREEN,
        ORANGE,
        BLUE,
        RED
    };

    int max = 1;

    for (int value : values) {
        max = Math.max(max, value);
    }

    int x = 35;

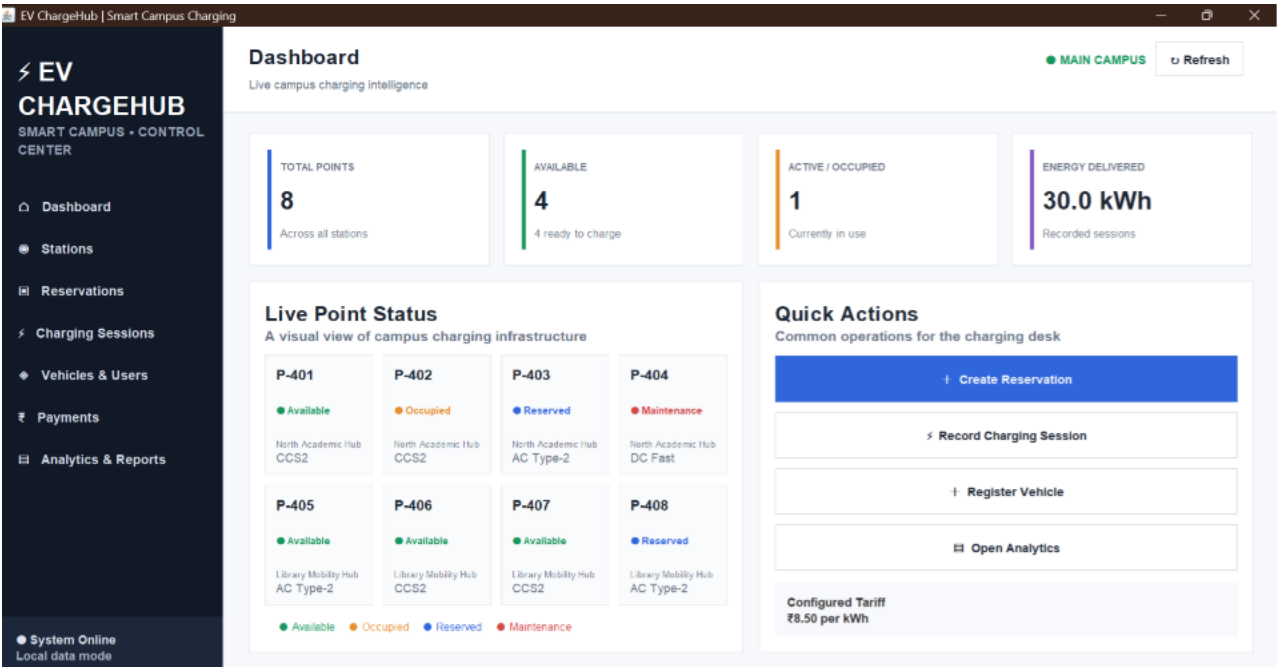
    for (int i = 0;
         i < values.length;
         i++) {

        int height =
            (int)
                (
                    (getHeight() - 90)
                        *
                        (
                            values[i]
                                /
                                (double) max

```

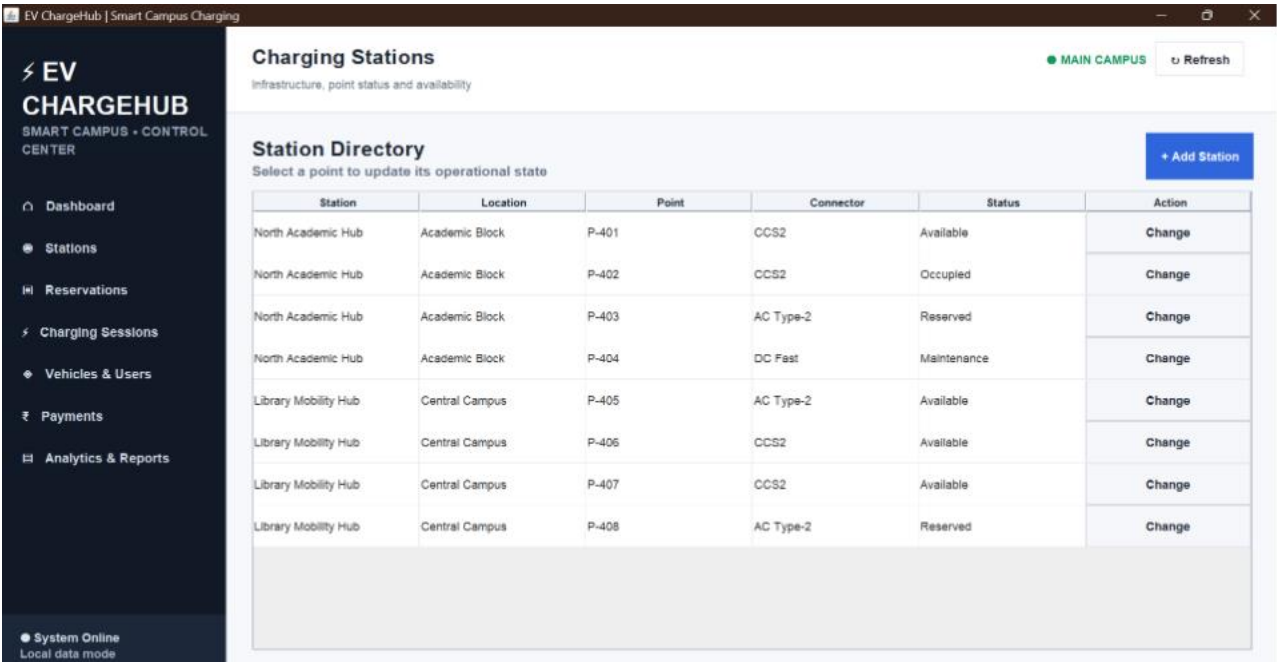
OUTPUT :

Figure 1. Dashboard



Main dashboard showing KPI cards, live charging-point status, quick actions and configured tariff.

Figure 2. Charging Stations



Station directory showing station, location, point, connector and operational status.

Figure 3. Charging Sessions

EV ChargeHub | Smart Campus Charging

MAIN CAMPUS Refresh

Charging Sessions
Track energy, duration and tariff-based cost

Charging Activity
Energy, duration and automatic cost calculation

+ Record Session

Session	Vehicle	Point	Energy	Duration	Cost	Status
S-601	KL-01-EV-7721	P-402	18.6 kWh	2.1 h	₹158.10	Completed
S-502	KL-07-EV-2410	P-405	11.4 kWh	1.4 h	₹96.90	Completed

Cost Formula: Energy Consumed * Tariff = kWh * ₹8.5

System Online
Local data mode

Charging activity table showing energy, duration, automatic cost and completion status.

Figure 4. Reservations

EV ChargeHub | Smart Campus Charging

MAIN CAMPUS Refresh

Reservations
Schedule charging slots without conflicts

Reservation Desk
Book an available point and create a traceable slot record

+ New Reservation

ID	Vehicle	Charging Point	Slot	Status
R-501	KL-07-EV-2410	P-403	Today - 02:30 PM	Confirmed
R-502	KL-08-EV-5016	P-408	Today - 04:00 PM	Confirmed

System Online
Local data mode

Reservation desk showing vehicle, charging point, slot and reservation status.

Figure 5. Vehicles & Users

EV CHARGEHUB
SMART CAMPUS - CONTROL CENTER

System Online
Local data mode

Vehicles & Users

Campus account and EV registration

MAIN CAMPUS Refresh

Campus Accounts & EVs

Identity and vehicle information used by reservations

+ Register User + Register Vehicle

User ID	Name	Email	Vehicle	Model	Registration
U-101	Ananya Menon	ananya@smartcampus.edu	EV-201	Tata Nexon EV	KL-07-EV-2410
U-102	Rahul Nair	rahul@smartcampus.edu	EV-202	MG ZS EV	KL-01-EV-7721
U-103	Meera Krishnan	meera@smartcampus.edu	EV-203	Hyundai Ioniq 5	KL-08-EV-5016

Campus accounts and EV registration view linking users with their vehicles.

Figure 6. Payments

EV CHARGEHUB
SMART CAMPUS - CONTROL CENTER

System Online
Local data mode

Payments

Charging transactions and payment status

MAIN CAMPUS Refresh

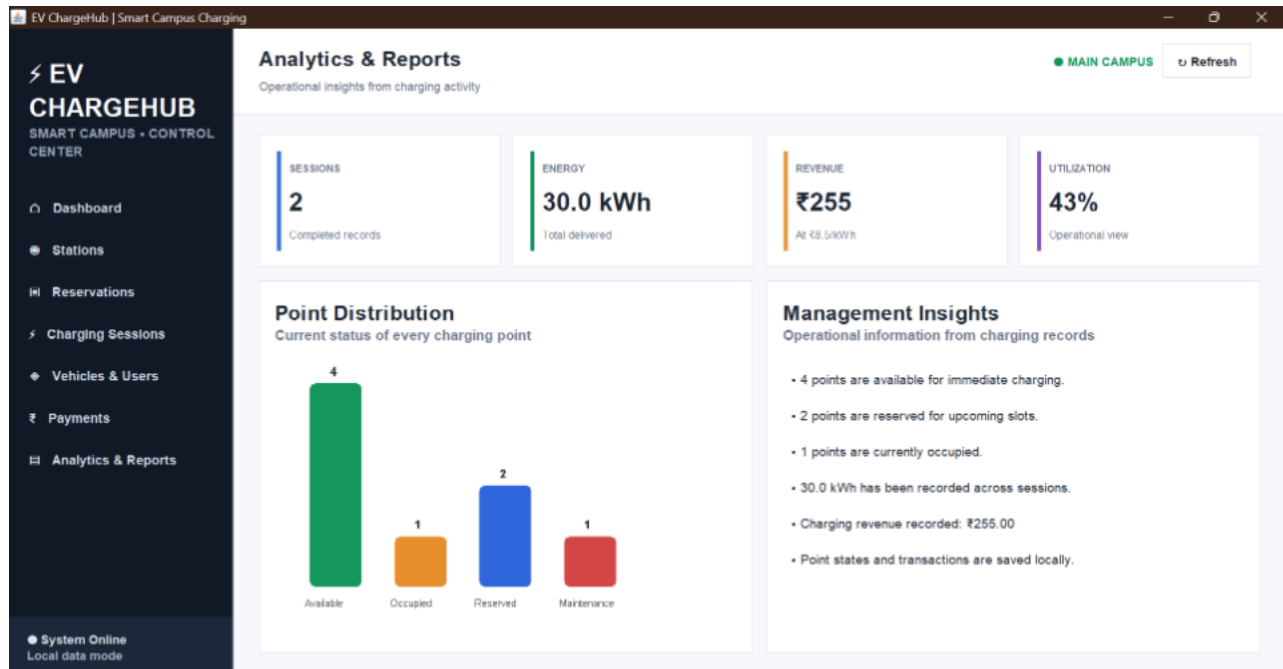
Payment Ledger

Transactions automatically created for completed sessions

Payment	Session	Vehicle	Amount	Status
PAY-701	S-601	KL-01-EV-7721	₹158.10	Paid
PAY-702	S-602	KL-07-EV-2410	₹96.90	Paid

Payment ledger showing payments automatically associated with completed charging sessions.

Figure 7. Analytics & Reports



Analytical dashboard showing session count, energy, revenue, utilization, point distribution and management insights.